

Базовый синтаксис

Операторы

```
ghci> 6 + 4 - 2
```

```
8
```

```
ghci> 3 * 8
```

```
24
```

```
ghci> 7 `div` 2
```

```
3
```

```
ghci> 7 `mod` 2
```

```
1
```

```
ghci> 2 ^ 10
```

```
1024
```

```
ghci> 2 ^ 10 ≤ 3 ^ 7
```

```
True
```

```
ghci> 1 < 2 && 1 < 3
```

```
True
```

```
ghci> 2 + 2 ≠ 5
```

```
True
```

```
ghci> 2 + 2 = 4 || 2 + 2 = 5
```

```
True
```

```
ghci> True && False
```

```
False
```

```
ghci> True || False
```

```
True
```

```
ghci> not False
```

```
True
```

Вызов функции

```
ghci> not True  
False  
ghci> negate 5  
-5
```

Вызов функции

```
ghci> min 3 8
```

```
3
```

```
ghci> min (3) (8)
```

```
3
```

Вызов функции

```
ghci> min 3 8
```

```
3
```

```
ghci> min (3) (8)
```

```
3
```

```
ghci> min 3 (max 8 1)
```

```
3
```

Вызов функции

```
ghci> min 3 8
```

```
3
```

```
ghci> min (3) (8)
```

```
3
```

```
ghci> min 3 (max 8 1)
```

```
3
```

```
ghci> max 3 max 8 1
```

```
<interactive>:6:1: error: [GHC-39999]
```

Вызов функции

```
ghci> min 3 8
```

```
3
```

```
ghci> min (3) (8)
```

```
3
```

```
ghci> min 3 (max 8 1)
```

```
3
```

```
ghci> (min 3 max) 8 1
```

```
<interactive>:6:1: error: [GHC-39999]
```

Infix / prefix

```
ghci> (+) 2 3
5
ghci> (==) 6 4
False
ghci> div 2 3
0
ghci> 2 `div` 3
0
ghci> mod 2 3
2
ghci> 2 `mod` 3
2
ghci> div 4 2 + 1
?
```

Infix / prefix

```
ghci> (+) 2 3
5
ghci> (==) 6 4
False
ghci> div 2 3
0
ghci> 2 `div` 3
0
ghci> mod 2 3
2
ghci> 2 `mod` 3
2
ghci> div 4 2 + 1
3
```

Infix / prefix

```
ghci> (+) 2 3
5
ghci> (==) 6 4
False
ghci> div 2 3
0
ghci> 2 `div` 3
0
ghci> mod 2 3
2
ghci> 2 `mod` 3
2
ghci> div 4 2 + 1
3
```

Правила:

1. ``f`` - вызов функции `f` в инфиксном стиле
2. `(op)` - вызов оператора в префиксном стиле
3. Скобки - важны

Ассоциативность

`infixl` : $a \circ b \circ c \circ d \equiv ((a \circ b) \circ c) \circ d$

`infixr` : $a \circ b \circ c \circ d \equiv a \circ (b \circ (c \circ d))$

`infix` : $a \circ b \circ c \circ d \equiv \text{error}$

`infixl` : $10 / 2 / 5 \equiv (10 / 2) / 5 = 1.0$

`infixr` : $2 \wedge 3 \wedge 2 \equiv 2 \wedge (3 \wedge 2) = 512$

Приоритеты

```
infixr 8 ^
```

```
infixl 7 *, /, `div`, `mod`
```

```
infixl 6 +, -
```

```
infix 4 =, /=, >, ≥, <, ≤
```

Приоритет - целое число в пределах $[0, 9]$

Применение функции - самый высокий приоритет (10)

:info

```
ghci> :i +
type Num :: * -> Constraint
class Num a where
  (+) :: a -> a -> a
  ...
      -- Defined in `GHC.Internal.Num'
```

```
infixl 6 +
```

```
ghci> :info /
type Fractional :: * -> Constraint
class Num a => Fractional a where
  (/) :: a -> a -> a
  ...
      -- Defined in `GHC.Internal.Real'
```

```
infixl 7 /
```

Write your own operator

```
(|-|) :: Int → Int → Int  
-- a (|-|) b = abs (a - b)  
(|-|) a b = abs (a - b)
```

Write your own operator

```
(|-|) :: Int → Int → Int  
-- a (|-|) b = abs (a - b)  
(|-|) a b = abs (a - b)
```

И это все?

Write your own operator

```
(|-|) :: Int → Int → Int  
-- a (|-|) b = abs (a - b)  
(|-|) a b = abs (a - b)
```

```
ghci> :i (|-|)  
(|-|) :: Int → Int → Int      -- Defined at app\Main.hs:4:1  
infixl 9 |-|  
  
ghci> 1 |-| 3 * 2  
4
```

Write your own operator

```
(|-|) :: Int → Int → Int  
-- a (|-|) b = abs (a - b)  
(|-|) a b = abs (a - b)  
infixl 6 |-|      -- fix
```

```
ghci> :r  
ghci> :i (|-|)  
(|-|) :: Int → Int → Int      -- Defined at app\Main.hs:4:1  
infixl 6 |-|  
  
ghci> 1 |-| 3 * 2  
5
```

Наименования операторов

Множество допустимых символов:

!, #, \$, %, &, *, +, ., /, <, =, >, ?, @, \, ^, |, -, ~, :

Примеры валидных наименований:

- ^_^
- <: .. :>
- /\ /\ /\ /\ /\ /\ /\ /\

Если оператор начинается с : - оператор является конструктором

Объявление функции

```
{-  
name  :: arg -> arg -> return -}  
mod   :: Int → Int → Int  
  
{-  
имя arg arg = function body    -}  
mod a    b    = a - b * a `div` b
```

Объявление функции

```
{-  
name  :: arg -> arg -> return -}  
mod   :: Int → Int → Int  
  
{-  
имя arg arg = function body    -}  
mod a    b    = a - b * a `div` b
```

```
-- корректное определение  
mod a  
    b = a - b * a `div` b  
  
-- parse error  
mod a  
b    = a - b * a `div` b
```

Частичное применение

pointful style

```
plus5 :: Int → Int  
plus5 a = a + 5
```

Частичное применение

pointful style

```
plus5 :: Int → Int  
plus5 a = a + 5
```

pointfree style

```
plus5 = (+) 5  
-- или так  
plus5 = (+5)
```

Частичное применение

Но иногда, нужно остановиться...

```
pentaSum a b c d e = a + b + c + d + e
```

Частичное применение

Но иногда, нужно остановиться...

```
pentaSum a b c d e = a + b + c + d + e
```

```
pentaSum = ((((((+)) .) . (+)) .) . (+)) .) . (+)
```

Использовать с осторожностью и не в лабах (пожалуйста, пожайте меня):
<https://pointfree.io/>

if expression

```
if <predicate>  
then <expression if True>  
else <expression if False>
```

```
factorial :: Integer → Integer -- Int vs Integer?  
factorial n = if n ≤ 1 then 1 else n * factorial (n - 1)
```

guards

```
grade :: Integer → String
grade n
  | n ≥ 90    = "A"
  | n ≥ 75    = "B"
  | n ≥ 60    = "C"
  | otherwise = "F"
```

guards

```
grade :: Integer → String
grade n
  | n ≥ 90    = "A"
  | n ≥ 75    = "B"
  | n ≥ 60    = "C"
  | otherwise = "F"
```

А что такое otherwise?

guards

```
grade :: Integer → String
grade n
  | n ≥ 90    = "A"
  | n ≥ 75    = "B"
  | n ≥ 60    = "C"
  | True     = "F"
```

otherwise = True

let expression

let <binding> in <expression>

```
dist :: Double → Double → Double → Double → Double
```

```
dist x1 y1 x2 y2 = let
```

```
    dx = x2 - x1
```

```
    dy = y2 - y1
```

```
in sqrt ( dx * dx + dy * dy)
```

let GHCi

```
ghci> let x = 5
ghci> x + 3
8
```

```
ghci> let y = 8 in y
8
ghci> y
<interactive>:32:1: error: [GHC-88464]
    Variable not in scope: y
```

where clause

<lhs> = <expression> where <binding>

```
dist :: Double → Double → Double → Double → Double
```

```
dist x1 y1 x2 y2 = sqrt (dx * dx + dy * dy)
```

```
  where
```

```
    dx = x2 - x1
```

```
    dy = y2 - y1
```

where vs let

Можно

```
dist x1 y1 x2 y2 =  
  sqrt  
    ( let dx = x2 - x1  
        dy = y2 - y1  
      in dx * dx + dy * dy  
    )
```

Нельзя

```
dist x1 y1 x2 y2 =  
  sqrt                                -- Parse error  
    (dx * dx + dy * dy where  
      dx = x2 - x1  
      dy = y2 - y1  
    )
```

where vs let

Можно

```
dist x1 y1 x2 y2 =  
  sqrt  
    ( let dx = x2 - x1  
        dy = y2 - y1  
      in dx * dx + dy * dy  
    )
```

```
f n  
  | n < 0      = neg  
  | otherwise = pos  
where  
  neg = ...  
  pos = ...
```

Нельзя

```
dist x1 y1 x2 y2 =  
  sqrt -- Parse error  
    (dx * dx + dy * dy where  
      dx = x2 - x1  
      dy = y2 - y1  
    )
```

```
f n =  
  let neg = ...  
      pos = ...  
  in -- после in ожидается выражение  
    | n < 0      = neg  
    | otherwise = pos
```

where vs let

Можно

```
dist x1 y1 x2 y2 =  
  sqrt  
    ( let dx = x2 - x1  
        dy = y2 - y1  
      in dx * dx + dy * dy  
    )
```

```
f n  
  | n < 0      = neg  
  | otherwise = pos  
where  
  neg = ...  
  pos = ...
```

Нельзя

```
dist x1 y1 x2 y2 =  
  sqrt -- Parse error  
    (dx * dx + dy * dy where  
      dx = x2 - x1  
      dy = y2 - y1  
    )
```

```
f n =  
  let neg = ...  
      pos = ...  
  in -- после in ожидается выражение  
    | n < 0      = neg  
    | otherwise = pos
```

Pattern matching

```
not :: Bool → Bool
```

```
not True  = False
```

```
not False = True
```

```
isZero :: Integer → Bool
```

```
isZero 0 = True
```

```
isZero _ = False
```

Pattern matching

```
not :: Bool → Bool
```

```
not True  = False
```

```
not False = True
```

```
isZero :: Integer → Bool
```

```
isZero 0 = True
```

```
isZero _ = False
```

```
-- что выведет при вызове isZero 0?
```

```
isZero :: Integer → Bool
```

```
isZero _ = False
```

```
isZero 0 = True
```

Pattern matching

```
-- 7!! = 7 * 5 * 3 * 1
-- 6!! = 6 * 4 * 2
doubleFactorial :: Integer → Integer
doubleFactorial 0 = 1
doubleFactorial 1 = 1
doubleFactorial n = n * doubleFactorial (n - 2)
```

Pattern matching

$$a_1 = 1, \quad a_2 = 2, \quad a_k = a_{k-1} + 3 * a_{k-2}$$

```
someSequence :: Integer → Integer  
someSequence n =
```

Pattern matching

$$a_1 = 1, \quad a_2 = 2, \quad a_k = a_{k-1} + 3 * a_{k-2}$$

```
someSequence :: Integer → Integer  
someSequence n = calcHelper n 1 2 where
```

Pattern matching

$$a_1 = 1, \quad a_2 = 2, \quad a_k = a_{k-1} + 3 * a_{k-2}$$

```
someSequence :: Integer → Integer
someSequence n = calcHelper n 1 2 where
  calcHelper 1 a1 _ = a1
  calcHelper 2 _ a2 = a2
```

Pattern matching

$$a_1 = 1, \quad a_2 = 2, \quad a_k = a_{k-1} + 3 * a_{k-2}$$

```
someSequence :: Integer -> Integer
someSequence n = calcHelper n 1 2 where
  calcHelper 1 a1 _ = a1
  calcHelper 2 _ a2 = a2
  calcHelper k x y  = calcHelper (k-1) y (y + 3 * x)
```

Pattern matching

$$a_1 = 1, \quad a_2 = 2, \quad a_k = a_{k-1} + 3 * a_{k-2}$$

```
someSequence :: Integer -> Integer
someSequence n = calcHelper n 1 2 where
  calcHelper 1 a1 _ = a1
  calcHelper 2 _ a2 = a2
  calcHelper k x y  = calcHelper (k-1) y (y + 3 * x)
```

```
ghci> someSequence 3
5
ghci> someSequence 33
95
```

:t / :type

```
ghci> :t someSequence  
someSequence :: Integer → Integer
```

```
ghci> :t not  
not :: Bool → Bool
```

```
ghci> :t id  
id :: a → a
```

:t / :type

```
ghci> :t someSequence  
someSequence :: Integer → Integer
```

```
ghci> :t not  
not :: Bool → Bool
```

```
ghci> :t id  
id :: a → a
```

```
id :: a → a  
id x = x
```

Параметрический полиморфизм

```
ghci> :t id  
id :: a → a
```

```
ghci> :t fst  
fst :: (a, b) → a
```

```
ghci> :t snd  
snd :: (a, b) → b
```

Параметрический полиморфизм

```
ghci> :t id
id :: a → a

ghci> :t fst
fst :: (a, b) → a

ghci> :t snd
snd :: (a, b) → b
```

Аналог generic / templates

Ad-hoc полиморфизм

```
ghci> :t (+)
(+) :: Num a => a -> a -> a

ghci> :t (==)
(==) :: Eq a => a -> a -> Bool
```

Ad-hoc полиморфизм

```
ghci> :t (+)
(+) :: Num a => a -> a -> a

ghci> :t (==)
(==) :: Eq a => a -> a -> Bool
```

- Ограничение контекста
- Экземпляр сам определяет реализацию
- Overloading?

Point-free и точка

$$(f \circ g) \ (x) = f \ (g(x))$$

Point-free и точка

$$(f \circ g) (x) = f (g(x))$$

$$f (x) = x^2$$

$$g (x) = x + 1$$

$$(f \circ g) (2) = (2 + 1) ^ 2 = 9$$

Point-free и точка

$(f \circ g) (x) = f (g(x))$

$f (x) = x^2$

$g (x) = x + 1$

$(f \circ g) (2) = (2 + 1) ^ 2 = 9$

```
f :: Integer → Integer
```

```
f x = x ^ 2
```

```
g :: Integer → Integer
```

```
g x = x + 1
```

```
-- (f . g)(2)
```

```
example :: Integer → Integer
```

```
example x = (f . g) x
```

Point-free и точка

```
(f ∘ g) (x) = f (g(x))  
f (x) = x^2  
g (x) = x + 1  
(f ∘ g) (2) = (2 + 1) ^ 2 = 9
```

```
f :: Integer → Integer  
f x = x ^ 2  
g :: Integer → Integer  
g x = x + 1  
  
-- (f ∘ g)(2)  
example :: Integer → Integer  
example x = (f . g) x
```

```
ghci> example 2  
9
```

Point-free и точка

```
(f ∘ g) (x) = f (g(x))  
f (x) = x^2  
g (x) = x + 1  
(f ∘ g) (2) = (2 + 1) ^ 2 = 9
```

```
f :: Integer → Integer  
f x = x ^ 2  
g :: Integer → Integer  
g x = x + 1  
  
-- (f ∘ g)(2)  
example :: Integer → Integer  
-- example x = (f . g) x  
example = f . g
```

```
ghci> composition 2  
9
```

Point-free и точка

$(f \circ g) (x) = f (g(x))$

$f (x) = x^2$

$g (x) = x + 1$

$(f \circ g) (2) = (2 + 1) ^ 2 = 9$

```
ghci> :t (.)  
(.) :: (b → c) → (a → b) → a → c
```

Point-free и точка

$(f \circ g) (x) = f (g(x))$

$f (x) = x^2$

$g (x) = x + 1$

$(f \circ g) (2) = (2 + 1) ^ 2 = 9$

```
ghci> :t (.)  
(.) :: (b -> c) -> (a -> b) -> a -> c
```

```
(.) :: (b -> c) -> (a -> b) -> a -> c  
(.) f g x = f (g (x))
```

Point-free и доллар

```
($) :: (a → b) → a → b  
f $ x = f x
```

Point-free и доллар

```
($) :: (a → b) → a → b  
f $ x = f x  
infixr 0 $
```

Point-free и доллар

```
($) :: (a → b) → a → b  
f $ x = f x  
infixr 0 $
```

... \$... \$... \$... \$...
⏟ ⏟ ⏟ ⏟ ⏟
5 4 3 2 1

Point-free и доллар

```
(.) :: (b → c) → (a → b) → a → c  
(.) f g x = f (g (x))
```

Point-free и доллар

```
(.) :: (b → c) → (a → b) → a → c  
(.) f g x = f (g (x))
```

```
(.) :: (b → c) → (a → b) → a → c  
(.) f g x = f $ g x
```

Point-free

```
f :: Integer → Integer  
f x = x ^ 2
```

```
ghci> f . f $ 2  
16  
ghci> f . f . f $ 2  
256
```

Point-free

```
f :: Integer → Integer  
f x = x ^ 2
```



```
f :: Integer → Integer  
f = (^ 2)
```

```
ghci> f . f $ 2  
16  
ghci> f . f . f $ 2  
256
```

```
ghci> f . f $ 2  
16  
ghci> f . f . f $ 2  
256
```

Point-free

```
f :: Integer → Integer  
f = (^ 2)
```

```
ghci> f . f $ 2  
16  
ghci> f . f . f $ 2  
256
```



```
ghci> (^2) . (^2) $ 2  
16  
ghci> (^2) . (^2) . (^2) $ 2  
256
```



ВСЁ!